Netty学以致用 之

简易聊天室开发

北京理工大学计算机学院
金旭亮

概述

- 在前面的课程中，我们已经学习了许多Netty的相关知识，这一讲，我们把学到的知识用于实际，写一个Netty聊天室程序。
- 在这个聊天室中，一群用户可以群发消息，彼此交换信息，当新用户加入，老用户退出时，其他用户都可以得到通知。

客户端技术方案

客户端主要完成两个功能：

1. 用户使用键盘输入信息，然后发给服务端。
2. 接收服务端传回的信息，显示在屏幕上。

客户端Handler

```
public class ChatClientHandler extends SimpleChannelInboundHandler<ByteBuf> {

    private ChannelHandlerContext ctx;

    @Override
    public void channelActive(ChannelHandlerContext ctx) {
        this.ctx = ctx;
    }

    @Override
    protected void channelRead0(ChannelHandlerContext ctx, ByteBuf msg) throws Exception {
        String serverMessage = msg.toString(CharsetUtil.UTF_8);
        System.out.println("\n<<< " + serverMessage);
        System.out.print(">>>");
    }

    //供外界调用，将一条消息发给服务器
    public void sendMessage(String msg) {
        //将消息发给服务器
        ctx.writeAndFlush(Unpooled.copiedBuffer(msg, CharsetUtil.UTF_8));
    }

    //供外界调用，关闭与服务器的连接
    public void close() {
        ctx.close();
    }

}
```

编写一个辅助类

```
//此类将等待用户在键盘输入一个字符串，然后敲回车
public class InputHelper {
    //接收用户输入
    public static String getUserInput(String prompt) throws IOException {
        var bufferReader = new BufferedReader(new InputStreamReader(System.in));
        String userInput = "";
        while (true) {
            System.out.print(prompt);
            userInput = bufferReader.readLine();
            if (userInput.isBlank() || userInput.isEmpty()) {
                System.out.println("不能发空消息!");
                continue;
            }
            //输入有效，退出循环
            return userInput;
        }
    }
}
```

客户端代码-1

```java
public class ChatClient {
    private final String host;  2 usages
    private final int port;  2 usages

    public ChatClient(String host, int port) {  1 usage
        this.host = host;
        this.port = port;
    }

    //客户端启动流程
    public void start() throws Exception {...}

    //启动一个单独的线程，接收用户输入：
    private static void handleUserInput(ChatClientHandler clientHandler) {...}

    public static void main(String[] args) throws Exception {
        new ChatClient( host: "127.0.0.1", ChatServer.SERVER_PORT).start();
    }
}
```

```java
//启动一个单独的线程，接收用户输入：
private static void handleUserInput(ChatClientHandler clientHandler) {  1 usage
    var userInputThread = new Thread(() -> {
        try {
            while (true) {
                String userInput = InputHelper.getUserInput( prompt: ">>>");
                if (userInput.equals("exit")) {
                    //退出并关闭通道
                    clientHandler.close();
                    break;
                } else {
                    //将消息发送出去
                    clientHandler.sendMessage(userInput);
                }
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    });
    userInputThread.setDaemon(true);
    userInputThread.start();
}
```

客户端代码-2

```
//客户端启动流程
public void start() throws Exception {
    EventLoopGroup group = new NioEventLoopGroup();
    var clientHandler = new ChatClientHandler();
    try {
        Bootstrap boot = new Bootstrap();
        boot.group(group)
                .channel(NioSocketChannel.class)
                .remoteAddress(new InetSocketAddress(host, port))
                .handler(new ChannelInitializer<SocketChannel>() {
                    @Override
                    protected void initChannel(SocketChannel ch) throws Exception {
                        ch.pipeline().addLast(clientHandler);
                    }
                });
        try {
            //连接服务器
            ChannelFuture f = boot.connect().sync();
            InetSocketAddress addr = (InetSocketAddress) f.channel().localAddress();
            System.out.println("成功连接服务器,监听: " + addr.getPort());
            //在独立的线程中接收用户输入
            handleUserInput(clientHandler);
            f.channel().closeFuture().sync();
        } catch (Exception ex) {
            System.out.println(ex.getMessage());
        }
    } finally {
        group.shutdownGracefully().sync();
    }
}
```

服务端技术方案

- 在服务端使用一个Channel集合来保存所有在线用户。
- 在服务端的Handler中，重写其channelActive方法，在此方法内，通知其他人：“有新用户上线了”，并且将新用户的Channel加入到集合中。
- 重写channelRead()方法，当收到消息时，通过遍历Channel集合，将消息转发给其他人。
- 重写channelInactive()方法，当有用户下线时，给其他人广播一条消息：“某用户下线”。

服务端Handler-1

```java
public class ChatServerHandler extends ChannelInboundHandlerAdapter {  1 usage
    //保存所有在线用户
    static Set<Channel> channelList = new HashSet<~>();  5 usages

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        //广播新客户端上线消息
        channelList.forEach(channel -> {
            String msg = "[客户端]" + ctx.channel().remoteAddress() + "上线了";
            ByteBuf buf = Unpooled.wrappedBuffer(msg.getBytes());
            channel.writeAndFlush(buf);
        });
        //新用户加入Channel集合
        channelList.add(ctx.channel());
        System.out.println("有客户端连接: " + ctx.channel().remoteAddress());
    }
```

服务端Handler-2

```java
@Override
public void channelInactive(ChannelHandlerContext ctx) throws Exception {
    //广播新客户端上线消息
    channelList.forEach(channel -> {
        if (channel != ctx.channel()) {
            String msg = "【客户端】" + ctx.channel().remoteAddress() + "下线。";
            ByteBuf buf = Unpooled.wrappedBuffer(msg.getBytes());
            channel.writeAndFlush(buf);
        }
    });
    channelList.remove(ctx.channel());
}
```

服务端Handler-3

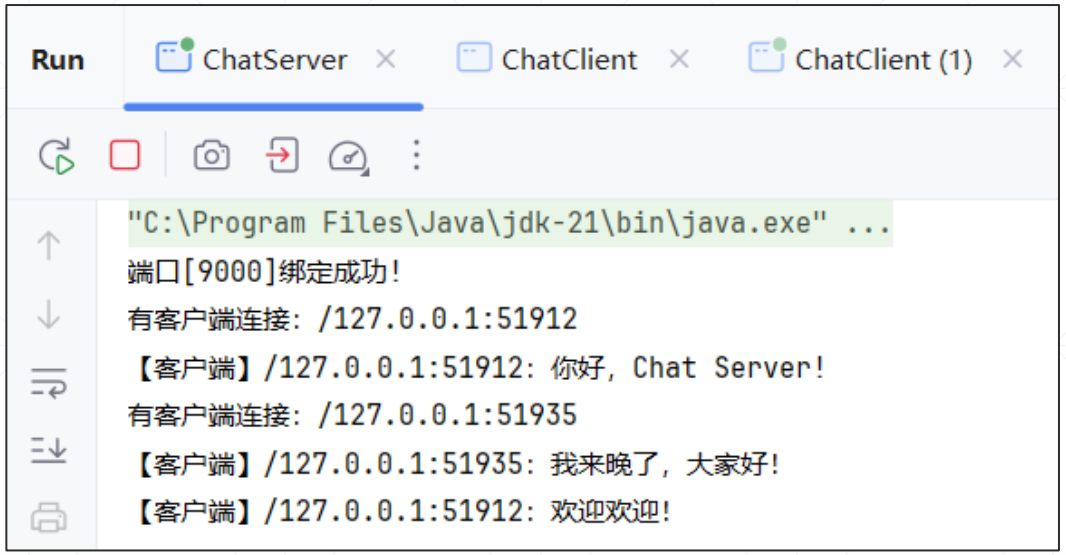
```
@Override
public void channelRead(ChannelHandlerContext ctx, Object msg)  {
    String message = "【客户端】" + ctx.channel().remoteAddress() + ": " +
            ((ByteBuf) msg).toString(StandardCharsets.UTF_8);
    System.out.println(message);
    //向聊天室的其他用户，广播消息
    channelList.forEach(channel -> {
        if (channel != ctx.channel()) {
            ByteBuf buf = Unpooled.wrappedBuffer(message.getBytes());
            channel.writeAndFlush(buf);
        }
    });
}
@Override
public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)  {
    System.out.println(cause.getMessage());
}
```

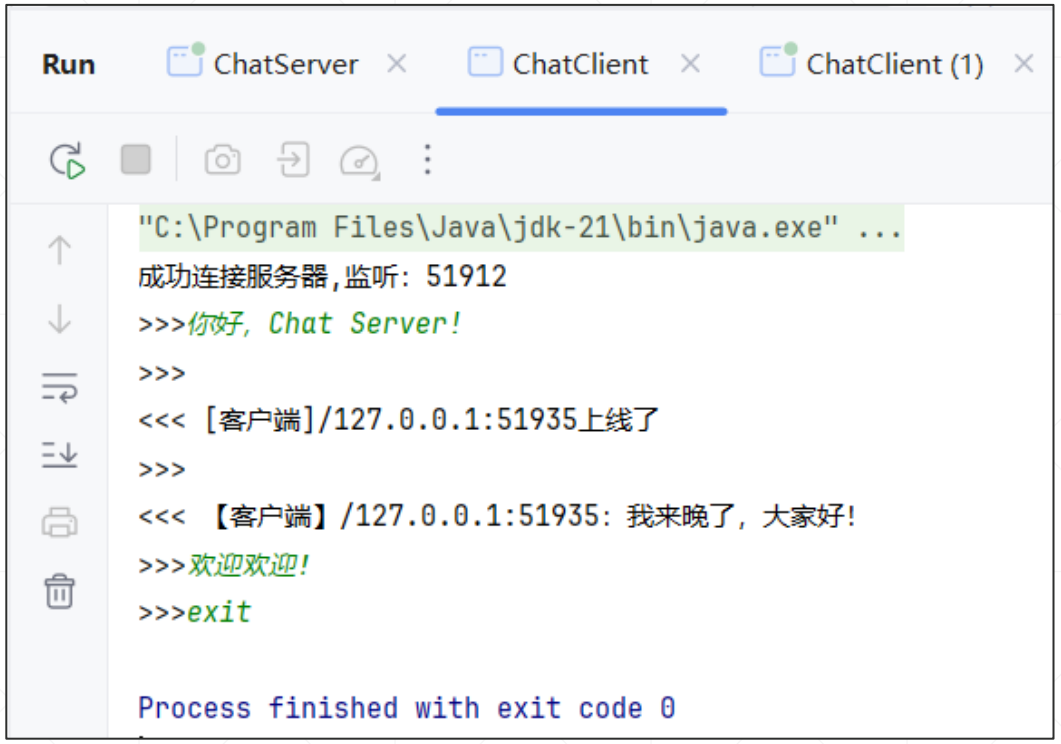
服务端启动代码

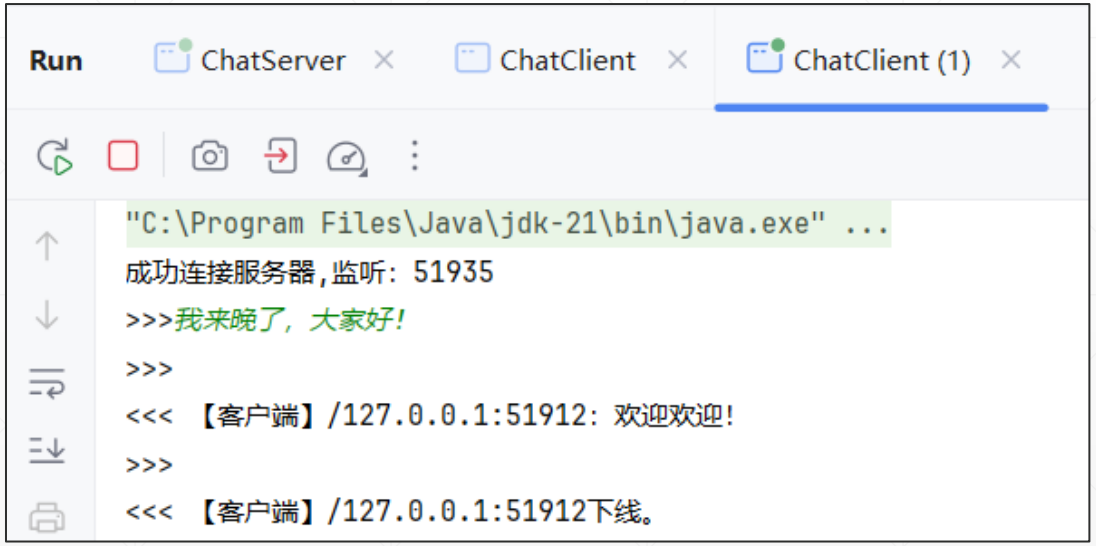
```
public class ChatServer {

    public static final int SERVER_PORT = 9000;  4 usages

    public static void main(String[] args) {
        NioEventLoopGroup boosGroup = new NioEventLoopGroup();
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();
        try {
            ServerBootstrap serverBootstrap = new ServerBootstrap();
            serverBootstrap.group(boosGroup, workerGroup)
                    .channel(NioServerSocketChannel.class)
                    .childHandler(new ChannelInitializer<SocketChannel>() {
                        @Override  2 usages
                        protected void initChannel(SocketChannel ch)  {
                            ch.pipeline().addLast(new ChatServerHandler());
                        }
                    });
            var bindResult = serverBootstrap.bind(SERVER_PORT).addListener(future -> {
                if (future.isSuccess()) {
                    System.out.println("端口[" + SERVER_PORT + "]绑定成功!");
                } else {
                    System.err.println("端口[" + SERVER_PORT + "]绑定失败!");
                }
            });
            bindResult.channel().closeFuture().sync();
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        } finally {
            boosGroup.shutdownGracefully();
            workerGroup.shutdownGracefully();
        }
    }
}
```

这些代码都是标准代码，没什么特殊的。

运行截图







两个客户端在线聊天，>>>表示用户输入，<<<表示从服务端接收到的信息。